

PRÁCTICA 6 – REFACTORIZACIONES

Ingeniería del Software II
3º Grado en Ingeniería Informática
Facultad de Ciencias – Universidad de Cantabria

Índice

1. Descripción del Sistema Heredado	3
2. Métricas Iniciales (Proyecto Original)	3
2.1. Reglas de cálculo aplicadas	3
2.2. Clase Transporte	3
2.3. Clase Conductor	4
2.4. Clase GestionTransportes	5
2.5. Clase GestionTransportesGUI	5
2.6. Clase CategoriaTransporte (enum)	6
2.7. Resumen métricas iniciales	6
3. Code Smells Detectados	6
4. Refactorizaciones Aplicadas	6
4.1. Refactorización 1: Replace Type Code with Subclasses	6
4.2. Refactorización 2: Extract Method (<code>extraSueldo()</code>)	7
4.3. Refactorización 3: Rename Class / Rename Method	7
4.4. Refactorización 4: Pull Up Field	7
4.5. Refactorización 5: Extract Method en <code>GestionTransportesGUI</code>	7
4.6. Corrección de bug (mejora adicional)	7
5. Métricas Finales (Proyecto Refactorizado)	8
5.1. Clase Transporte (abstracta)	8
5.2. Clase TransportePersonas	8
5.3. Clase TransporteMercancias	8
5.4. Clase TransporteMercanciasPeligrosas	9
5.5. Clase Conductor (refactorizada)	9
5.6. Clase GestionTransportesGUI (refactorizada)	9
5.7. Resumen métricas finales	10
6. Análisis Comparativo de Métricas	10
6.1. Comparación directa	10

6.2. Interpretación	10
-------------------------------	----

1. Descripción del Sistema Heredado

La aplicación gestiona conductores y transportes de una empresa. Cada conductor tiene datos personales (DNI como identificador, nombre, apellidos, dirección) y una lista de transportes realizados. El sueldo se calcula como:

$$\text{Sueldo} = 700\text{€} + \sum_{\text{transportes}} (5\text{€/hora} + \text{extra por tipo})$$

Los tipos de transporte y sus extras son:

- **Personas:** +0,5/hora (no colectivo, < 10 personas) o +1/hora (colectivo, ≥ 10)
- **Mercancías:** +2/tonelada
- **Mercancías Peligrosas:** +2/tonelada +50 fijo por transporte

2. Métricas Iniciales (Proyecto Original)

2.1. Reglas de cálculo aplicadas

- **CC** (Complejidad Ciclomática): nodos de condición +1. Cada **case** de un **switch** suma 1; cada **||** o **&&** adicional en una condición suma 1.
- **CCog** (Complejidad Cognitiva): +1 por estructura de control; +*N* adicional por nivel de anidación (*N* = nivel actual); **else/else if** suman siempre 1; **switch** suma 1; secuencias de **||** o **&&** iguales suman 1 por secuencia.
- **WMC** = $\sum$ CC de todos los métodos de la clase; **WMCn** = WMC / número de métodos.
- **CBO**: número de clases distintas del sistema con las que una clase está acoplada ($\text{AFF} \cup \text{EFF}$, sin duplicar).
- **DIT**: longitud del camino hasta la raíz de la jerarquía de herencia.
- **NOC**: número de subclases inmediatas.

2.2. Clase Transporte

WMCn = $9/5 = 1,80$ **CCogn** = $3/5 = 0,60$ **CBO** = 1 (CategoriaTransporte)
DIT = 0 **NOC** = 0

Cálculo CC del constructor:

```
if (horas <= 0 || valor <= 0 || cat == null)  -> +1(if) +1(|| adic
    .) +1(|| adic.) = 3 nodos
if (cat.equals(...))                        -> +1(if)
Total: 1 + 4 = 5
```

Método	CC	CCog
Transporte(double, CategoriaTransporte, int)	5	4
horas()	1	0
categoria()	1	0
ton()	1	0
getPersonas()	1	0
TOTAL	WMC = 9	CCog = 3

Cuadro 1: Métricas de Transporte (original)

Cálculo CCog del constructor:

```

if (...||...||...) -> +1(if) +1(una sola secuencia de ||) = 2
if (cat.equals)      -> +1(if, nivel 0)                      = 1
CCog = 3

```

2.3. Clase Conductor

Método	CC	CCog
Conductor(String,...)	5	2
dni()	1	0
getDni()	1	0
getNombre()	1	0
getApellido1()	1	0
apellido2()	1	0
getDire()	1	0
suelo()	6	7
anhadeTransporte(Transporte)	1	0
TOTAL	WMC = 18	CCog = 9

Cuadro 2: Métricas de Conductor (original)

$WMCn = 18/9 = 2,00$ $CCogn = 9/9 = 1,00$ $CBO = 2$ (Transporte, CategoriaTransporte)
 $DIT = 0$ $NOC = 0$

Cálculo CC del constructor:

```

if (dni==null || nombre==null || apellido1==null || direccion==null
)
-> +1(if) +1(|| adic.) +1(|| adic.) +1(|| adic.) = 4 nodos
Total: 1 + 4 = 5

```

Cálculo CCog del constructor:

```

if (...||...||...||...) -> +1(if) +1(una sola secuencia de ||) = 2
CCog = 2

```

Cálculo CC de sueldo():

```

for (...)                -> +1
case Mercancias           -> +1
case MercanciasPeligrosas -> +1
case Personas             -> +1
if (t.getPersonas() < 10) -> +1
Total: 1 + 5 = 6

```

Cálculo CCog de sueldo():

```

for(nivel 0)                -> +1           = 1
switch(nivel 1)             -> +1+1(anid)    = 2
if(nivel 2, dentro de case Personas) -> +1+2(anid) = 3
else                        -> +1 (siempre) = 1
CCog = 1+2+3+1 = 7

```

2.4. Clase GestionTransportes

Método	CC	CCog
buscaConductor(String)	3	3
anhadeConductor(...)	2	1
conductores()	1	0
TOTAL	WMC = 6	CCog = 4

Cuadro 3: Métricas de GestionTransportes (original)

$WMCn = 6/3 = 2,00$ $CCogn = 4/3 = 1,33$ $CBO = 1$ (Conductor) $DIT = 0$
 $NOC = 0$

Cálculo CCog de buscaConductor():

```

for(nivel 0)  -> +1           = 1
if(nivel 1)   -> +1+1(anid) = 2
CCog = 3

```

2.5. Clase GestionTransportesGUI

Método	CC	CCog
main(String[])	15	20
mensaje(String, String)	1	0
TOTAL	WMC = 16	CCog = 20

Cuadro 4: Métricas de GestionTransportesGUI (original)

$WMCn = 16/2 = 8,00$ $CCogn = 20/2 = 10,00 \leftarrow muy\ elevado$ $CBO = 5$ $DIT = 0$ $NOC = 0$

2.6. Clase CategoriaTransporte (enum)

WMC = 0 (sin métodos de lógica) CBO = 0 DIT = 0 NOC = 0

2.7. Resumen métricas iniciales

Clase	WMC	WMCn	CCog	CCogn	CBO	DIT	NOC
Transporte	9	1,80	3	0,60	1	0	0
Conductor	18	2,00	9	1,00	2	0	0
GestionTransportes	6	2,00	4	1,33	1	0	0
GestionTransportesGUI	16	8,00	20	10,00	5	0	0
CategoriaTransporte	0	—	0	—	0	0	0

Cuadro 5: Resumen de métricas del proyecto original

3. Code Smells Detectados

1. **Abuso de switch** (`Conductor.sueldo()`): el `switch` sobre `CategoriaTransporte` viola el principio Open/Closed. Cada nuevo tipo de transporte requiere modificar `Conductor`.
2. **Nombre de clase incorrecto** (`gestionTransportes`): no sigue la convención Java (PascalCase).
3. **Inconsistencia en getters** (`Conductor`): `dni()` y `apellido2()` no siguen la convención `get*`.
4. **Método main muy largo** (`GestionTransportesGUI`): más de 80 líneas en un solo método, `CCog` = 20.
5. **Bug en GestionTransportesGUI**: imprime `conductor.getNombre()` dos veces en lugar de nombre y apellido.

4. Refactorizaciones Aplicadas

4.1. Refactorización 1: Replace Type Code with Subclasses

Code smell: `switch` sobre `CategoriaTransporte` en `Conductor.sueldo()`.

Se elimina el enum `CategoriaTransporte` y se crea la siguiente jerarquía de clases:

- `Transporte` (abstracta) con método abstracto `extraSueldo()`
- `TransportePersonas` `extends` `Transporte`
- `TransporteMercancias` `extends` `Transporte`

- `TransporteMercanciasPeligrosas` extends `TransporteMercancias`

El método `Conductor.sueldo()` pasa de `CC=6/CCog=6` a `CC=2/CCog=1`, usando polimorfismo en lugar del `switch`.

4.2. Refactorización 2: Extract Method (`extraSueldo()`)

Code smell: lógica de cálculo del extra mezclada con el bucle en `sueldo()`.

Cada subclase encapsula su propia lógica de cálculo en el método `extraSueldo()`, que es llamado polimórficamente desde `Conductor.sueldo()`.

4.3. Refactorización 3: Rename Class / Rename Method

Code smell: `gestionTransportes` no sigue las convenciones Java.

Se renombra a `GestionTransportes`. Adicionalmente se añade `getApellido2()` manteniendo compatibilidad con `apellido2()`.

4.4. Refactorización 4: Pull Up Field

Code smell: el atributo `ton` sería duplicado entre `TransporteMercancias` y `TransporteMercanciasPeligrosas`

El campo `ton` se declara `protected` en `TransporteMercancias` y `TransporteMercanciasPeligrosas` lo hereda directamente.

4.5. Refactorización 5: Extract Method en `GestionTransportesGUI`

Code smell: método `main` excesivamente largo (`CCog=20`).

Se extraen cuatro métodos auxiliares:

- `procesarNuevoConductor()`
- `procesarNuevoTransporte()`
- `mostrarSueldoConductor()`
- `mostrarMejorConductor()`

4.6. Corrección de bug (mejora adicional)

En `GestionTransportesGUI` original se imprimía el nombre dos veces:

```
// BUG original
msj += conductor.getNombre() + " " + conductor.getNombre() + "\n";

// Correccion
```

```
msj += conductor.getNombre() + " " + conductor.getApellido1() + "\n";
```

5. Métricas Finales (Proyecto Refactorizado)

5.1. Clase Transporte (abstracta)

Método	CC	CCog
Transporte(double)	2	1
horas()	1	0
extraSueldo() (abstracto)	1	0
TOTAL	WMC = 4	CCog = 1

Cuadro 6: Métricas de Transporte (refactorizado)

$WMCn = 4/3 = 1,33$ $CCogn = 1/3 = 0,33$ $CBO = 0$ $DIT = 0$ $NOC = 2$

5.2. Clase TransportePersonas

Método	CC	CCog
TransportePersonas(double, int)	2	1
getPersonas()	1	0
extraSueldo()	2	1
TOTAL	WMC = 5	CCog = 2

Cuadro 7: Métricas de TransportePersonas

$WMCn = 5/3 = 1,67$ $CCogn = 2/3 = 0,67$ $CBO = 0$ $DIT = 1$ $NOC = 0$

5.3. Clase TransporteMercancias

Método	CC	CCog
TransporteMercancias(double, int)	2	1
getTon()	1	0
extraSueldo()	1	0
TOTAL	WMC = 4	CCog = 1

Cuadro 8: Métricas de TransporteMercancias

$WMCn = 4/3 = 1,33$ $CCogn = 1/3 = 0,33$ $CBO = 0$ $DIT = 1$ $NOC = 1$

5.4. Clase TransporteMercanciasPeligrosas

Método	CC	CCog
TransporteMercanciasPeligrosas(double, int)	1	0
extraSueldo()	1	0
TOTAL	WMC = 2	CCog = 0

Cuadro 9: Métricas de TransporteMercanciasPeligrosas

$$\text{WMCn} = 2/2 = 1,00 \quad \text{CCogn} = 0/2 = 0,00 \quad \text{CBO} = 0 \quad \text{DIT} = 2 \quad \text{NOC} = 0$$

5.5. Clase Conductor (refactorizada)

Método	CC	CCog
Conductor(String,...)	5	4
dni()	1	0
getDni()	1	0
getNombre()	1	0
getApellido1()	1	0
apellido2()	1	0
getApellido2()	1	0
getDire()	1	0
sueldo()	2	1
anhadeTransporte(Transporte)	1	0
TOTAL	WMC = 15	CCog = 5

Cuadro 10: Métricas de Conductor (refactorizado)

$$\text{WMCn} = 15/10 = 1,50 \quad \text{CCogn} = 5/10 = 0,50 \quad \text{CBO} = 1 \text{ (Transporte)} \quad \text{DIT} = 0 \quad \text{NOC} = 0$$

5.6. Clase GestionTransportesGUI (refactorizada)

$$\text{WMCn} = 24/6 = 4,00 \text{ (antes 8,00)} \quad \text{CCogn} = 19/6 = 3,17 \text{ (antes 10,00)} \quad \text{CBO} = 7 \quad \text{DIT} = 0 \quad \text{NOC} = 0$$

Método	CC	CCog
main(String[])	8	7
procesarNuevoConductor(...)	2	1
procesarNuevoTransporte(...)	5	4
mostrarSueldoConductor(...)	2	1
mostrarMejorConductor(...)	6	6
mensaje(String, String)	1	0
TOTAL	WMC = 24	CCog = 19

Cuadro 11: Métricas de GestionTransportesGUI (refactorizado)

Clase	WMC	WMCn	CCog	CCogn	CBO	DIT	NOC
Transporte (abs.)	4	1,33	1	0,33	0	0	2
TransportePersonas	5	1,67	2	0,67	0	1	0
TransporteMercancias	4	1,33	1	0,33	0	1	1
TransporteMercanciasPeligrosas	2	1,00	0	0,00	0	2	0
Conductor	15	1,50	5	0,50	1	0	0
GestionTransportes	6	2,00	4	1,33	1	0	0
GestionTransportesGUI	24	4,00	19	3,17	7	0	0

Cuadro 12: Resumen de métricas del proyecto refactorizado

Clase / Métrica	WMCn antes	WMCn después	CCogn antes	CCogn después
Transporte / jerarquía	1,80	1,38 (media)	0,60	0,33 (media)
Conductor.sueldo()	CC=6, CCog=7	CC=2, CCog=1	—	—
GestionTransportesGUI	8,00	4,00	10,00	3,17

Cuadro 13: Comparación de métricas antes y después de refactorizar

5.7. Resumen métricas finales

6. Análisis Comparativo de Métricas

6.1. Comparación directa

6.2. Interpretación

Refactorización 1 (Replace Type Code with Subclasses): es la de mayor impacto. El método `Conductor.sueldo()` pasa de CC=6/CCog=7 a CC=2/CCog=1, una reducción del 67–86 %. Esto refleja directamente la eliminación del `switch`, que es el code smell más grave del sistema original. La complejidad se distribuye ahora entre las subclases de `Transporte`, con valores bajos y homogéneos ($CCog \leq 1$ por método).

Refactorización 5 (Extract Method en GUI): reduce el WMCn de `GestionTransportesGUI` de 8,00 a 4,00 y el CCogn de 10,00 a 3,17. Aunque el WMC total sube (más métodos), los métodos individuales son más pequeños, legibles y mantenibles. Este es un resultado

típico y esperado del Extract Method.

DIT y NOC reflejan la nueva jerarquía: **Transporte** tiene NOC=2, **TransporteMercancias** tiene NOC=1, y las hojas tienen DIT=1 o 2. Valores bajos de DIT indican que la jerarquía no es excesivamente profunda, lo que es positivo para la mantenibilidad.

CBO de Conductor se reduce de 2 a 1: ya no depende de **CategoriaTransporte** (que desaparece), solo de la clase abstracta **Transporte**. Esto refleja el menor acoplamiento obtenido.

En conclusión, las métricas confirman de manera objetiva las mejoras de calidad: menor complejidad en los métodos clave, distribución más uniforme de la lógica entre clases bien definidas, y mejor separación de responsabilidades.